Plexus Discovery

Progress note — written to be read, not decoded

31 July 2026

Plain English throughout. Organised by phase, because phases are where we stop and you decide.
The technical specification is preserved separately in `plexus2_technical.pdf`.

---

1. What we are doing — two things at once

Track A — build the method. An automated loop that searches for biological *mechanisms* rather than parameter values. It proposes a change to the model's structure ("remove the pushing force", "add oriented cell division"), writes down what it expects *before* running, runs the simulation, measures, and records whether it was right. The discipline that makes it worth building: a change of numbers can never masquerade as a new idea.

Track B — reproduce Okuda's result, as the proof that Track A works. Okuda et al. (2018) grew a hollow ball of cells that spontaneously forms tubes, undulations and branches, driven by a chemical pattern on its own surface. We are rebuilding that here. If the loop finds the mechanism largely by itself, the method is demonstrated. *The figure is not a side quest — it is the evidence.*

The two tracks feed each other: the reproduction is the honest test of the method, and the method is what makes the reproduction more than hand-tuning.

2. Where we actually are

Track A is further along than Track B, and until this morning I was over-reporting both.

An independent review this morning found that **three of the measurements the loop relies on are broken**, and you have since found a fourth. Not the simulations — the *rulers*. A large part of what I reported over the last two days was measured with faulty instruments and does not currently stand up.

The clearest example, and why you were right to stop me.
I ran 32 simulations overnight, sweeping five settings, and reported a strong conclusion: *"making the chemistry shape the tissue and keeping the tissue intact are mutually exclusive."* Every one of those 32 runs ended with exactly **1778 cells**. I noticed that, wrote it down, and called it "remarkable". I never asked the obvious question: *why is it identical every time?*
The answer is arithmetic. The memory buffer for the mesh holds 3552 points, and the geometry of a closed cell sheet fixes the largest possible number of cells at $(3552 + 4)/2 = 1778$. **Every run hit the wall and stopped.** I was not measuring biology. I was measuring the size of an array.
That conclusion is now **withdrawn**. It may still turn out to be true — but it is not measured, and I presented it as if it were.

3. What we believe about the biology

Nothing. The register is empty.

Every measurement we have was taken with at least one of the four broken rulers in the path. There is no scientific claim in this document, and there should not be one until Phase 1 has re-measured with instruments that work.

An earlier draft of this section had a table grading five claims as "solid", "weak" or "unsupported". Cedric was right to strike it, and the reason is worth recording.

Grading claims on a confidence scale is the correct move when the instruments are sound and the evidence is thin. It is the *wrong* move when the instruments themselves are broken — then the register should be *voided*, not annotated. Grading felt like rigour, but it smuggled the conclusions forward: a reader skims "solid", carries it, and the broken ruler is forgotten two pages later. If we cannot say how wrong a number is, it does not get a grade; it gets deleted.

What we do know — about our own code, not about biology

These are facts established by *reading the source*, not by measuring anything. They are a different kind of statement, and they belong in a different list.

- **In the two front-page movies, the chemistry cannot affect the shape.** The switch that lets the chemical drive growth is set to 50, and the chemical is a quantity of order 1. The switch term is therefore about 10^{-9} — off for any value the chemical could take. Separately, the growth setting is at its uniform limit, so every cell grows identically regardless. Both are visible in the configuration file; neither needed an experiment.
- **We were never running Okuda's system** (§8).
- **The mesh buffer is sized as a multiple of the starting cell count**, so a run that begins with 150 cells can never exceed 1778. That is why all 32 runs stopped there.

None of these tells us anything about tubes. They tell us the experiment we thought we were running is not the experiment that ran.

4. The four broken rulers

All four are *measurement* faults. The physics may be fine; we simply could not see it. The fourth is the camera, and you spotted it by looking at the movies — it is the ruler the automatic eye-check reads, and it is broken in two independent ways.

1. **The "does the shape survive?" test was measuring a fresh sphere.** The intent: grow the tube, switch off growth and pushing, let it relax, see what survives. A real tube survives; an artificially forced one collapses. The code was meant to continue from the end of the run — instead it started a brand-new simulation from the initial sphere, and so measured the starting condition every time. It returned essentially the same value (1.014) in 14 of 16 runs. **That value carries full weight in the score that ranks every experiment.** *Not yet fixed — one of Phase 0's open items.*

2. **Two different quantities were both called "how far it sticks out".** One measured distance from the centre *of the tissue*; the other from the *origin of the coordinate system.* The ball drifts sideways as it grows, so the second reads drift as elongation. *Fixed this morning.*

3. **A safety check existed, but not where the danger was.** We once chased a phantom result for days, caused by pairing mismatched frames. A guard was written to catch it. My overnight study used exactly the pattern that guard forbids, in three places, and never called the guard. *Fixed this morning.*

4. **The camera hides the thing we are looking for, and hides growth entirely.** *(Your observation, confirmed in the code.)* Two separate faults, both in the movie every automatic eye-check reads:

   - **One fixed viewpoint.** The campaign's movies are drawn from a single angle that never moves. A tube growing away from that angle sits *behind the body* and is simply not in the picture. There is no second view — no top-down — to catch it.
   - **The zoom follows the object.** The frame is re-fitted to the tissue on *every frame*, so a ball that doubles in size is drawn the same size throughout. **Growth is invisible by construction.** Nothing that watches these movies — human or machine — can see the thing the whole campaign is about.

   Okuda's figures use a fixed frame and a consistent viewpoint, which is exactly why they read clearly. *Not fixed — now part of Phase 0, because it is an instrument, not a nicety.*

5. What is genuinely working

The corrections should not hide the parts that earned their keep.

- **Predictions are written down before runs, and wrong ones are recorded as wrong.** Twice in two days I proposed a mechanism and the measurement killed it within the hour. Both sit in the record as retractions rather than being quietly edited away. This is the part I would defend hardest.
- **The video check found a disagreement — but I over-claimed who was right.** On the first full round the system described each movie in words automatically and compared that to the numbers. It *vetoed the two highest-scoring runs*, including one that three separate analyses had called a tube. I reported this as "the numbers were wrong and the picture was right". **That was too strong.** Given ruler 4 above, the picture may have been hiding a tube behind the body, or failing to show growth at all. What the check genuinely established is that *the two*

instruments disagree — which is still valuable, and is precisely why it exists. Which one to believe is not yet decided, and cannot be until the camera is fixed.

- **The loop refuses to guess.** Asked to score 32 results against predictions written in a unit it did not recognise, it declined 32 times rather than inventing 32 confirmations.

6. The grey cells — a broken mesh that nothing stops

You saw grey faces in the videos where cells should be white or red, and read it as the cell representation breaking. I went through the code. **You are right, and the situation is worse than not-noticing: it is noticed, blended into an average, and then ignored.**

Why grey. A cell is drawn as a little block: a coloured top face and darker side walls, the sides shaded to 72% brightness so they only show at the silhouette. An unactivated cell is white, so *its side walls are grey.* Seeing grey head-on means you are looking at the side of a cell where its top should be — the cell has folded over, collapsed, or turned inside out.

1. Does the system see it? *Partly, and in the worst possible way.* There is a measurement, and it lumps three completely different problems into one number: caps that have folded over; slivers so thin they are barely cells (often harmless — a cell that just divided); and cells with fewer than three neighbours, which is **genuinely broken topology**, a cell that is no longer a cell. All three land in a single figure. A reading of "20% flagged" could be twenty percent of mildly bent caps or twenty percent of destroyed cells, and nothing distinguishes them.

Worse, a proper validity test *exists in this codebase* and is used by the two-dimensional runners — and was never wired into the three-dimensional path the entire campaign runs on. That is now the third time we have found a guard that was written and then not installed where the danger was.

2. Does it care? **No.** That number is recorded and nothing acts on it. In my overnight study one setting flagged **84% of cells**. It still produced a protrusion number, that number was still scored, still ranked, and still went into my conclusions. Nothing anywhere refused it.

3. Does it do anything about it? **No.** There is no check when the topology actually changes — not after a cell divides, not after neighbours swap. The validation plan specifies exactly these checks. They were never built.

4. What should happen, and who decides? Both specification and code, and the specification first, because the definition is the hard part.

- **Separate the three failure modes.** A folded cap, a fresh sliver and a cell with two neighbours are not the same event and must never again share an average.
- **Check where the topology changes**, not at render time. By the time you can see it, the physics has already run on a broken mesh for hundreds of frames.
- **Truncate the run at the breakage; do not throw it away.** My first version of this rule said a broken mesh makes the whole run not-evidence. Cedric was right that this is too strong: if the mesh fails at frame 380 of 400, the first 379 frames are perfectly good physics. The rule is therefore an **evidence horizon** — the first frame at which the mesh stops being valid. Every measurement is taken over frames *before* that point; "final" means the last valid frame, not the last frame; and the horizon is recorded alongside the run ("valid to 380 of 400") so a reader can

see how much of it counted. A run that breaks at frame 5 tells us nothing about shape — but it is still a real result about *stability*, and it should be recorded as one rather than discarded.

An over-strict rule is its own failure mode: a rule that throws away ninety-five percent of a good run will quietly stop being applied.

- **Who:** the **Metrologist** owns the standard, since it owns the validation plan and is the only role that can retract things already recorded. The **Critic** applies it — free, automatic, before anything is scored. Not the eye-check: by the time grey is visible on a video, the run is already paid for.

The consequence Cedric's correction exposes — the score rewards breaking the mesh. This is the part that matters most, and it only became visible once we stopped asking "is the run valid" and started asking "valid *until when*".

The headline measurement, "how far does it stick out", is recorded as the **largest value over the whole run**. Nothing restricts that maximum to frames where the mesh was still intact. When a mesh tears apart, cells fly outward and that number spikes. *The spike becomes the score.*

So the search is not merely tolerating broken meshes — **it is being rewarded for producing them**. The most elongated run in the overnight study scored 32.7 with 84% of its cells broken; the elongation *was* the destruction. Any search left running long enough under this rule will discover that the cheapest way to score well is to blow the tissue apart, and will report it as the best mechanism found.

Truncating at the evidence horizon fixes this at the root: the maximum is taken over valid frames only, so destruction earns nothing.

One subtlety that truncation alone does not fix. If one run is valid to frame 400 and another to frame 200, their "final" states are at *different moments*, and comparing them compares two different times rather than two different mechanisms. A batch comparison must therefore be made at a **common frame** — the earliest horizon in the batch — and say so. My rho sweep compared final values across runs that almost certainly broke at different times, and truncation on its own would not have saved it.

Done today: the curves are now readable, and they already say a lot. Two changes, both small. The per-frame measurements are now saved in the same format as the mechanical ones, so a single loader reads both — they were in two different formats, which is part of why nothing read them. And the *shape* of each curve is now classified automatically, by arithmetic rather than judgement, into one of: flat, still-rising, settled, peaked-then-declined, blown-up, or **held against a hard limit**.

That last one is the 1778 case. A settling quantity still wobbles in its last few samples; a quantity pinned against a ceiling repeats *identically*, sample after sample. One line of arithmetic separates them, and it would have caught the overnight study on its first run.

Applied to all 24 archived runs:

What the trajectories say	Runs
Cell count **still rising** at the final frame — the run was stopped, not finished, so its "final" value is just where we happened to halt	**18 / 24**
Mesh quality **still degrading** at the final frame	15 / 24
Tube diameter **peaked mid-run then declined** — so the recorded final value reports the decay rather than the phenomenon	**10 / 24**
Cell count **pinned against a hard ceiling**	4 / 24
Mesh degrades somewhere before the end	18 / 24

Two things are worth singling out. The classifier independently flagged one run as pinned — and it is the same run the type-checker had separately rejected for overflowing its buffer. Two unrelated detectors agreeing on real data is the best evidence the classifier works. And three runs come out as trustworthy for only the first three-to-five percent of their length, while sitting in the archive as results.

The caveat is load-bearing and I will not bury it: those percentages rest on the blended number, so a run marked "5% trustworthy" may be five percent broken cells or five percent harmless slivers. They are a list of runs to *re-examine*, not a verdict on any of them. Separating the three failure modes is what turns this from a flag into a measurement.

The retroactive part. If broken cells were present in the archived runs — and the 84% figure says they were in at least one — then the runs behind the withdrawn conclusion were computed on partly-destroyed meshes. That is a *second, independent* reason Phase 1 must re-measure, and because the three failure modes were blended, the recorded numbers cannot tell us how bad it was. We have to re-run with them separated.

7. The team of agents — what is actually switched on

You were told 6 of 12 are implemented or used. I audited it directly against the code rather than accept the figure, and it is better than that — but there are two real gaps, and they are worth closing exactly as you say.

The design calls for 15 roles (14 automatic, plus a human who patches the substrate from outside the loop). **All 15 are written. 13 actually run.** The two that do not:

Role	What it is for	Status
Evolution	Take the round's winner and improve it — simplify it, combine it with a runner-up, ground it in the paper. Without it the loop can only ever *pick* from what it happened to propose; it can never *refine*.	Written, **never called anywhere**.
Referee	Rank a batch by a proper tournament (every candidate against every other, aggregated) rather than by sorting on one number. The tournament is what protects against ordering bias.	Written, but **only ever exercised in its own self-test**. The live round sorts on a single score instead.

The other thirteen — the one that holds the objective, the one that reads the papers, the one that proposes, the peer-reviewer, the type-checker, the second-opinion judge, the duplicate detector, the three independent analysts, the eye-check, the one that writes the causal story, the pattern-distiller, and the one that certifies the instruments — all ran on the last round.

Two honest caveats: the eye-check is currently reading a broken camera (ruler 4), and the time budget is not merely unenforced — it is not even measured.

How long the agents take — nobody knows, and the arithmetic is alarming

You asked whether the time per call is reasonable, and suggested five minutes a round as the target. Checking it produced a short answer and a long one.

The short answer: we do not measure it. The last round ended by reporting `calls: 0, minutes: 0.0` — for a round that made about twenty-five calls. A time ledger exists, and no agent is routed through it. So the declared ceiling of 25 minutes has never constrained anything, and there is no record of how long any call has ever taken.

The long answer: the declared allowances add up to nearly two hours. These are timeouts rather than expected durations, but they are what the system would permit before intervening, on a round with five scored runs:

Role	Allowance	Calls per round	Worst case
Reads the papers	4 min	1	4 min
Proposes the batch	10 min	1	10 min
Peer-reviews it	5 min	1	5 min
Analysts	4 min	**15** (3 per run)	**60 min**
Eye-check	3 min	5	15 min
Second opinion	2 min	1	2 min
Writes the story	6 min	2	12 min
Distils the patterns	8 min	1	8 min
		27 calls	**116 min**

Why five minutes is nonetheless achievable. Twenty of the twenty-seven calls are the per-run ones — three analysts and an eye-check, for each of five runs — and they are completely independent of one another. They run one after another today for no reason other than that the code was written as a loop. Run them together and that whole block costs the time of its slowest single call rather than the sum of twenty.

Adding the change already on the list — starting the next round's proposal *while* the current simulations are still running, so its ten minutes leave the critical path entirely — a round's unavoidable serial cost is roughly: peer-review, the slowest analyst, the second opinion, two stories in parallel, and the distillation. **About four to five minutes, which is your number.**

Two items follow, and they are separated deliberately: *measuring* is cheap and goes into Phase 0 with the other instruments; *making it fast* is real work and goes into Phase 2b.

8. What stands between us and the Okuda figure

I previously framed this as an exotic parameter-range problem. **Cedric's reading is closer to right: most of the gap is mundane, and it is about the starting conditions.** That matters, because mundane gaps get closed in a day.

The mundane part — and it is most of it

Starting condition	Okuda	What we ran	Fix
Number of cells at the start	200, 2000, 4000	**150**	set the number
Room to grow into	grows from there	**capped at 1778**	bigger buffer
Number of spots at the start	∼5 at 2000 cells	never counted	count them
Size of those spots	set by his figures	never measured	measure them

We were not running his system. The overnight study began with 150 cells — his smallest case starts with 200 and his undulation case with 2000. And because the mesh buffer is sized as a multiple of the *starting* count, beginning at 150 capped us at 1778 cells. His 4000-cell case is not reachable at all today. None of this is subtle physics; it is four numbers.

The same applies to the pattern. Rather than trying to copy his length-scale setting — which in our code is not even the same quantity as his — **we match what he reports seeing**: about five spots on a 2000-cell ball. Tune our diffusion until we get five spots at 2000 cells, then freeze it. That is a calibration against an observation, and it sidesteps the units problem entirely.

What is left after that — in full

Three things remain once the starting conditions are right. Two are numbers; one is not.

(a) How sharply a cell decides to grow. Each cell reads the local chemical concentration and decides how fast to grow. The question is how *abruptly* it decides. One number controls that. At the low end the response is gentle and proportional: a cell with a little chemical grows a little, a cell with a lot grows a lot, and every cell in the tissue is growing to some degree. At the high end it becomes almost a switch: below a threshold a cell does nothing at all, above it the cell grows at full rate, and there is almost nothing in between.

Okuda uses 10. Our search is allowed up to 8. That sounds like a small shortfall, but the behaviour is very nonlinear in this number and the difference is not cosmetic — it is the difference between a bulge and a tube.

Here is why. With a gentle response, the whole ball inflates and the chemical spot merely inflates slightly faster than its surroundings: you get a broad, soft swelling with no clear edge. With a sharp switch, the tissue divides into two populations — growing and not growing — separated by a crisp boundary. *That boundary is the neck.* A tube is not simply a region that grew; it is a region that grew while the tissue immediately around it stayed put and held the base narrow. Without a sharp switch there is nothing to hold the base narrow, and a tube relaxes into a dome.

Fix: widen one range. Half an hour of work.

(b) How far the inhibitor travels compared with the activator. The surface pattern is made by two chemicals. One is self-reinforcing and short-range — it builds up locally and makes more of itself. The other suppresses it and travels further. Wherever the first wins locally you get a spot; the second spreads out around that spot and stops another forming too close. **The ratio of the two travel distances sets the spacing of the spots**, and therefore how many appear on a ball of a given size.

Okuda's inhibitor spreads with a value of 10 against the activator's 1. **Our search allows at most 2.** With a ratio that low the inhibitor barely outruns the activator, which puts us close to the edge of where any pattern forms at all — and where one does form, it is the wrong pattern: many small spots crowded together, or a fine speckle, instead of a few well-separated domains.

This is why we cannot currently produce his roughly five spots on a two-thousand-cell ball. It also means **this is the number we calibrate**, rather than copying: turn it up until we count five spots at two thousand cells, then freeze it and never touch it again. Matching a thing he *observed* is safe; matching a parameter value across two differently-scaled models is not.

Fix: widen one range, then calibrate against a spot count. Half a day.

(c) The real gap: the tissue's shape does not feed back into the chemistry. The coupling between chemistry and shape is supposed to run in both directions. Ours runs in one.

Chemistry to shape works: a cell with a lot of chemical grows, the tissue deforms. That half is built and it is what produces the bulges we have seen.

Shape to chemistry is absent. Chemical moves between neighbouring cells, and in Okuda's model the amount that moves depends on the geometry — how much wall two cells actually share, and how much volume the receiving cell has to dilute it into. Our code does something much cruder: each cell simply **averages its neighbours**, counting each one equally. Two cells sharing a thin sliver of wall exchange exactly as much chemical as two sharing a broad face. A cell stretched to twice its volume dilutes its contents exactly as if it had not stretched at all.

The consequence is that **deformation is invisible to the chemistry**. The pattern sits on the tissue like a decal: the tissue can bend, stretch and grow underneath it, and the pattern carries on as though nothing happened.

Why this probably does not block a plain tube. A tube needs one thing from the chemistry: a spot that stays put and stays on. A decal does that perfectly well. So we may get tubes without touching this.

Why it almost certainly blocks branching. Branching, in Okuda's account, is a consequence of the feedback we are missing. A tube grows; the cells at its tip stretch and their volumes change; that dilutes the chemical unevenly across the tip; the single chemical domain splits into two; the growth zone splits with it; and the tube forks. *Every step of that chain runs through shape affecting chemistry.* With the feedback missing, a tube in our model can grow indefinitely but has no mechanism by which it could ever decide to divide. We would not get branching, and — worse — we would not know whether that was a real result or our missing arrow.

Fix — and it belongs in the pre-flight, not later. Write the flow between two cells so it is weighted by their shared wall area and divided by the receiving cell's volume. The mesh already knows both quantities; they are simply not being used.

This is a *second implementation of a contract we already have*, not a new concept: same set of things it acts on, same kind of operation, same family. So no decision about the language is needed — it is exactly the split between "what a mechanism is" and "how it is realised" working as intended, in the same way that three different chemistries already sit behind one reaction contract.

The payoff is bigger than the fix. Today the diffusion operator has only one implementation, and the code does not even offer swapping it as a move — so the loop could not test this even if it wanted to. Register the geometric version alongside the blind one and mark the swap as a real mechanism change, and the necessity experiment becomes a *single legal move the loop can propose by itself*: run the same composition with geometry-aware flow and with geometry-blind flow, and see which phenotypes survive. That is the ablation that produces the claim Okuda never makes — *branching requires this arrow* — and it is expressible in the search language rather than being something we run by hand.

Then: write it, test it against the case where it must agree with the old one (all walls equal, all volumes equal), promote it into the shared codebase with its declarations, and re-run the spot calibration — changing how the chemical spreads changes how far it spreads.

In short: two numbers to widen, one of which we then calibrate against something Okuda actually reported — and one missing arrow, which we can probably reach a tube without, but certainly cannot reach branching without.

9. The phases

Eight stages. **I stop at the end of each one and wait for you.** That is the partition.

Phase 0 — Pre-flight: fix the instruments IN PROGRESS **(2 of 6)**

Goal. Make the measurements trustworthy. Nothing runs unattended until this lands.
Why. Running before this writes uninterpretable results into a permanent record. Four rulers were wrong; two are now right.
Remaining.

- **The "does the shape survive?" test** (§4, item 1). Either repair it so it continues from the end state, or remove it from the score. A constant carrying full weight is worse than no term at all.
- **The camera** (§4, item 4). Fix the frame so it does not follow the object — one size, held for the whole run and shared across runs, as in Okuda's figures — and add a second viewpoint so a tube cannot hide behind the body. This was originally scheduled much later as a presentation nicety. It is not: it is the ruler the eye-check reads, and until it is fixed we cannot say whether the eye-check's vetoes were right.
- **Start from Okuda's cell counts** (§8). Set the starting number to his, and size the buffer for where the run is going rather than where it began. Four numbers.
- **Put a stopwatch on every agent call** (§7). Route them through the existing time ledger so the next round reports what it actually cost. Measuring only — speeding it up is Phase 2b.
- **Remove the silent parameter caps.** The search is confined to hand-written ranges by a clamp it is never told about, and the same ranges are not enforced on us at all. *This is the 1778 bug again*: an invisible limit that turns "we did not look there" into "there is nothing there". Where a real stability limit exists, derive it from the timestep and the mesh and make breaching it **loud** — not evidence, like a saturated buffer. A bound is either physical and derived, or it does not exist.
- **Write the missing shape-to-chemistry coupling, test it, and promote it** (§8c). Okuda published this mechanism; implementing it is reproduction, not discovery, so we build it rather than waiting for the loop to invent it.
- **Check that the cells are still cells** (§6). Broken cells are currently detected only as a blended number and never block anything. Separate the failure modes, check them where the topology changes, and make a broken mesh *not evidence*.

What you get. A one-line confirmation that the rulers are sound, plus one before/after movie so you can see the camera change with your own eyes.

Phase 1 — Re-measure what Phase 0 invalidated NOT STARTED (~1 hour)

Goal. Re-run the 32-simulation study with a buffer ten times larger and the frame-pairing guard live.
Why. This is the phase that decides whether my overnight conclusion was real.
What it will decide. One of two outcomes, and *both are good*:

- it survives — we have a genuine structural result, which is exactly what a paper about the method wants; or
- it dissolves — and we avoided publishing an artefact.

What you get. The answer in a sentence, plus the corrected table.

Phase 2 — Make Okuda's settings reachable NOT STARTED (~1 week)

Goal. Widen the three out-of-range settings; replace the mislabelled length-scale with the quantity Okuda actually varies; add the missing shape-to-chemistry feedback; add a fixture that asserts his published constants rather than tuning them.
Why. Without this, Track B cannot succeed at any effort level.
What you get. A list of what now matches his paper and what still does not, with reasons.

Phase 2b — Switch on the two agents that never run NOT STARTED (1 day)

Goal. Put **Evolution** and the **Referee** into the live round (§7); run the twenty per-run calls together instead of one after another; move the proposal off the critical path by starting it while the simulations run. Target: **a round costs five minutes of agent time**, measured, not asserted.

Why. Without Evolution the loop can only pick among the changes it happened to think of, never improve the best one — which is half of what makes it a search rather than a lottery. Without the Referee's tournament, ranking rests on a single number, and that number is the one Phase 0 is currently repairing.

Also: make ablation compulsory, and make quantification somebody's job. A causal claim needs two things and we currently enforce neither. *Ablation* answers "is this mechanism necessary" — run the identical model with the mechanism removed and see whether the phenotype dies. *A sweep of the operator's main parameter* answers "how much does it matter" — a dose–response curve instead of an assertion. Either alone is an opinion. The precedent for enforcing it already exists: the control is not left to the proposer's taste, it is *mandated* — slot 0, or the proposal is rejected. Ablation gets the same treatment. The duties split across agents that already exist:

Agent	New duty
Proposer	Must *offer* the with/without pair for any mechanism it makes a causal claim about — an obligation, like the control slot, not a preference.
Critic	**Rejects the batch** if a causal claim arrives without its ablation. A deterministic check, free, before any compute is spent.
Reflection	Judges whether it is the *right* ablation — are you removing the thing your claim is actually about? That needs taste, so it needs a model.
Evolution	Owns *quantification*: sweep the winner's main parameter to get the curve. This also switches on one of the two idle agents.
Interpreter	**May not write "causes"** without both. Only the additive direction ran? Then the word is "sufficient".
Metrologist	Owns the standard, and retracts recorded claims that fail it.

Two deliberate exclusions: the Proposer must not be its own enforcer (the author does not referee), and the Supervisor does not own this — it holds the objective, not the method.

What you get. Confirmation that all fourteen automatic roles ran in one round, the measured wall-clock against the five-minute target, and a batch that was rejected for asserting cause without testing necessity.

Phase 3 — Measurements that can tell his regimes apart NOT STARTED (2 days)

Goal. Four measurements: how many spots, how thick the tube, how many branches over time, and the shape of the surface. Calibrate against his roughly five spots at 2000 cells, then freeze the calibration.

Why. We currently have no measurement that can distinguish a tube from an undulation from a branch — precisely the three outcomes the figure has to show.

What you get. The four numbers, checked against his published figures.

Phase 3b — Point the camera *along the tube* NOT STARTED **(half a day)**

Goal. Phase 0 makes the camera honest — fixed frame, second viewpoint. This phase makes it *smart*: lock it to the tube's own axis, one view side-on and one looking straight down the tube.
Why. The end-on view *is* the thickness and branch-count measurement, so camera and measurement share one axis. It is a refinement, not a repair — the repair is in Phase 0.
What you get. Movies in which the tube is always in full profile, whichever way it grew.

Phase 4 — The demonstration NOT STARTED **(one wave, ∼1 hour)**

Goal. Run the designed grid — six to eight simulations — that produces the figure: tube, undulation and branching, at his settings.
What you get. *The figure.*

Phase 5 — The method claim NOT STARTED

Goal. Write up what the loop found by itself versus what we told it, with the record of predictions and retractions as the evidence.
What you get. The argument that Track A works, resting on Track B.

10. What I need from you

Nothing blocking. Two things worth knowing:

1. **Phase 1 is the interesting one** and it costs about an hour of compute. When it finishes I will bring you the answer, not the reasoning — unless you want the reasoning, in which case say so and it becomes the default.
2. **Tell me if this note is the right shape.** Too long, too short, wrong order — it is cheap to change, and it is the thing that lets you steer.

Appendix — words I could not avoid

Only these four. Everything else above is ordinary English.

- **Cell sheet (or mesh)** — the model of the tissue: a hollow ball of polygonal cells sharing walls, like a football.
- **Operator** — one mechanism as a reusable building block: "cells divide", "chemical spreads", "surface pulls tight". A model is a combination of these.
- **The loop** — the automated cycle: propose a change, predict the outcome, run, measure, record whether the prediction held.
- **Phase** — one of the eight stages above. Our agreed stopping points.

Technical detail — the control law, the agent roles, the operator language, the validation ladder and the full external review — is preserved in `plexus2_technical.pdf` (23 pp). My working log is `SESSION_LOG.md`. You should not need either.